Atividade 3 – Matheus Milanezi

1. Caracterize seu ambiente de experimentação:

- Sistema operacional (host): Windows 10 Pro, com Docker Desktop (backend WSL2).
- Virtualização: containers Docker (Ubuntu 24.04), orquestrados via Docker Compose. Cada container representa um "host" da topologia (r1, c1, c2).
- Interfaces de rede / MTU: `eth0` em cada container, MTU 1500, conectadas a uma rede `lab-net` (bridge Docker) com IPv6 habilitado.
- Ferramentas instaladas na imagem: `iproute2`, `iputils-ping`, `radvd`, `tcpdump`, `iperf3`, `procps`.
- Topologia adotada:

```
[c1] ----\
          eth0 -- [r1 / lab-net bridge]
[c2] ----/
```

Assim como no ambiente de referência do roteiro (namespaces + bridge `br-lab`), os três hosts compartilham o mesmo segmento L2; r1 não é um "meio de caminho" ponto-a-ponto, e sim o membro da rede que roteia/anuncia o prefixo.

Evidência real (rodada de `lab/run-lab.sh`):

```
PS D:\Servidor\Faculdade\UNISINOS\2026 2\redes de computadores\Atividade 3\lab\logs> Get-Content 01-ambiente.log
--- docker compose ps ---
NAME       IMAGE     COMMAND            SERVICE   CREATED         STATUS                  PORTS
lab-c1-1   lab-c1    "sleep infinity"   c1        2 seconds ago   Up Less than a second
lab-c2-1   lab-c2    "sleep infinity"   c2        2 seconds ago   Up Less than a second
lab-r1-1   lab-r1    "sleep infinity"   r1        2 seconds ago   Up Less than a second

--- r1: ip -6 addr show dev eth0 ---
2: eth0@if15: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default  link-netnsid 0
    inet6 fd00:dead:beef::4/64 scope global nodad
       valid_lft forever preferred_lft forever
    inet6 fe80::9875:e2ff:fee2:bd92/64 scope link tentative
       valid_lft forever preferred_lft forever
--- r1: MTU ---
mtu 1500

--- c1: ip -6 addr show dev eth0 ---
2: eth0@if13: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default  link-netnsid 0
    inet6 fd00:dead:beef::2/64 scope global nodad
       valid_lft forever preferred_lft forever
    inet6 fe80::88db:a2ff:fe8d:eafd/64 scope link
       valid_lft forever preferred_lft forever
--- c1: MTU ---
mtu 1500

--- c2: ip -6 addr show dev eth0 ---
2: eth0@if14: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default  link-netnsid 0
    inet6 fd00:dead:beef::3/64 scope global nodad
       valid_lft forever preferred_lft forever
    inet6 fe80::3087:e9ff:fef0:ff4b/64 scope link
       valid_lft forever preferred_lft forever
--- c2: MTU ---
mtu 1500
```

MTU confirmado em 1500 nas três interfaces `eth0` (padrão do driver bridge do Docker).

2. Monte a rede IPv6 virtual e capture os pacotes

2.1 Ambiente (Docker Compose)

Estrutura final em lab/: imagem única (`Dockerfile`, Ubuntu 24.04 com iproute2/radvd/tcpdump/iperf3), `docker-compose.yml` com os três serviços (r1, c1, c2) na rede `lab-net`, e um script de orquestração (`run-lab.sh`) que roda e loga cada etapa em `lab/logs/`.

Rede Docker configurada com `enable_ipv6: true` e dois subnets: `172.28.0.0/24` (IPv4) e `fd00:dead:beef::/64` (IPv6 — usado deliberadamente diferente do prefixo do laboratório `fd00:1234:5678::/64`, exatamente para poder distinguir visualmente "endereço que o Docker atribuiu (IPAM)" de "endereço que veio do SLAAC via radvd").

`cap_add`/`sysctls` usados e o motivo:

- r1: `NET_ADMIN` + `NET_RAW` (necessários para configurar endereço, rodar `radvd` e capturar com `tcpdump`); `net.ipv6.conf.all.forwarding: 1` (roteador precisa encaminhar pacotes entre hosts);

`net.ipv6.conf.all.accept_ra: 2` (sem efeito prático nesta topologia, pois r1 não tem roteador acima dele, mantido por padrão defensivo).

- c1 / c2: `NET_ADMIN` + `NET_RAW` (para `ping`/`tcpdump` local); `net.ipv6.conf.all.accept_ra: 2`, testado e confirmado que, sem `forwarding` ligado nesses hosts, isso não era estritamente necessário (o padrão `accept_ra=1` já aceitaria RA); mantido como precaução contra o kernel ignorar RA.

2.2 Link-local:

Endereços link-local (fe80::/10) de c1 e c2, gerados automaticamente ao subir a interface, antes de qualquer Router Advertisement:

```
PS D:\Servidor\Faculdade\UNISINOS\2026 2\redes de computadores\Atividade 3\lab\logs> Get-Content 02-linklocal.log
--- c1 ---
2: eth0@if13: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default  link-netnsid 0
    inet6 fe80::88db:a2ff:fe8d:eafd/64 scope link
       valid_lft forever preferred_lft forever
--- c2 ---
2: eth0@if14: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default  link-netnsid 0
    inet6 fe80::3087:e9ff:fef0:ff4b/64 scope link
       valid_lft forever preferred_lft forever
PS D:\Servidor\Faculdade\UNISINOS\2026 2\redes de computadores\Atividade 3\lab\logs>
```

Teste do `%eth0`: ping no link-local de c2 a partir de c1 sem e com o identificador de zona.

Sem `%eth0`:

```
Windows PowerShell
PS D:\Servidor\Faculdade\UNISINOS\2026 2\redes de computadores\Atividade 3\lab> docker compose exec c1 ping -6 -c 2 fe80::3087:e9ff:fef0:ff4b
ping: Warning: IPv6 link-local address on ICMP datagram socket may require ifname or scope-id => use: address%<ifname|scope-id>
PING fe80::3087:e9ff:fef0:ff4b (fe80::3087:e9ff:fef0:ff4b) 56 data bytes

--- fe80::3087:e9ff:fef0:ff4b ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1029ms
```

Com `%eth0`:

```
Windows PowerShell
PS D:\Servidor\Faculdade\UNISINOS\2026 2\redes de computadores\Atividade 3\lab> docker compose exec c1 ping -6 -c 2 "fe80::3087:e9ff:fef0:ff4b%eth0"
PING fe80::3087:e9ff:fef0:ff4b%eth0 (fe80::3087:e9ff:fef0:ff4b%eth0) 56 data bytes
64 bytes from fe80::3087:e9ff:fef0:ff4b%eth0: icmp_seq=1 ttl=64 time=0.263 ms
64 bytes from fe80::3087:e9ff:fef0:ff4b%eth0: icmp_seq=2 ttl=64 time=0.138 ms

--- fe80::3087:e9ff:fef0:ff4b%eth0 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1018ms
rtt min/avg/max/mdev = 0.138/0.200/0.263/0.062 ms
```

2.3 Roteador + SLAAC:

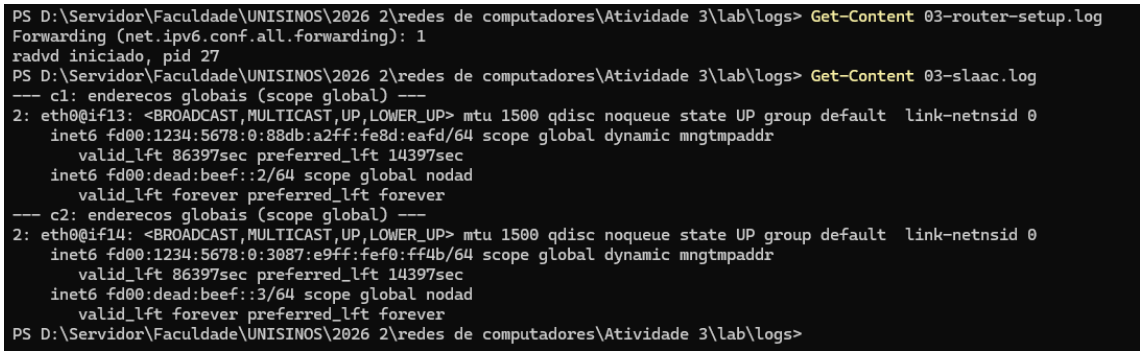
Comandos executados em r1 (via `scripts/setup-router.sh`): `ip -6 addr add fd00:1234:5678::1/64 dev eth0`, confirmação de `forwarding=1`, e `radvd -C /etc/radvd.conf`. Conteúdo do `radvd.conf`:

```
interface eth0 {
    AdvSendAdvert on;
    AdvManagedFlag off;
    AdvOtherConfigFlag off;
    prefix fd00:1234:5678::/64 {
        AdvOnLink on;
        AdvAutonomous on;
    };
};
```

Endereços globais criados em c1 e c2 após o RA:

```
c1: fd00:1234:5678:0:88db:a2ff:fe8d:eafd/64  scope global dynamic mngtmpaddr
c2: fd00:1234:5678:0:3087:e9ff:fef0:ff4b/64  scope global dynamic mngtmpaddr
```

Validação de que é SLAAC de verdade: Cada interface ficou com dois endereços globais, o do IPAM do Docker (`fd00:dead:beef::2` em c1, `::3` em c2, sem flags especiais, `nodad`) e o do prefixo do laboratório com as flags `dynamic mngtmpaddr`, que é a marcação textual do próprio kernel Linux. Essa distinção só foi possível porque escolhemos, de propósito, um subnet do IPAM do Docker diferente do prefixo do laboratório validado antes num teste mínimo isolado (r1+c1).



2.4 Captura de Router Advertisement e Neighbor Discovery

Captura feita com `tcpdump -i eth0 -n -vv icmp6 -w /logs/icmpv6-capture.pcap` em r1, salva em `lab/logs/icmpv6-capture.pcap` (abrir no Wireshark com filtro `icmpv6.type == 134 || icmpv6.type == 135 || icmpv6.type == 136`).

Trecho relevante do RA capturado (`tcpdump -vv`, também conferido de forma independente no Wireshark):

```
fe80::9875:e2ff:fee2:bd92 > ff02::1: ICMP6, router advertisement, length 56

  hop limit 64, Flags [none], pref medium, router lifetime 1800s

  prefix info option (3): fd00:1234:5678::/64, Flags [onlink, auto], valid time 86400s, pref. time 14400s
```

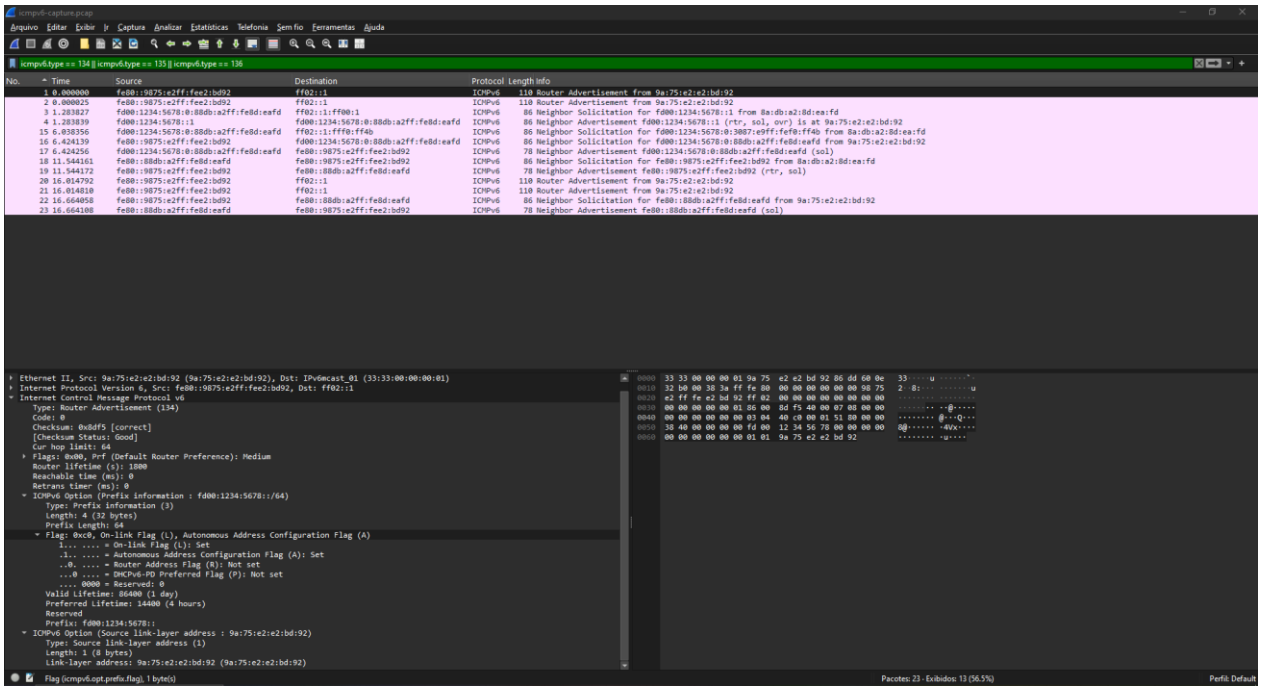
No Wireshark, o mesmo pacote detalha: `Flag: 0xc0, On-link Flag (L): Set`, `Autonomous Address Configuration Flag (A): Set`, `Router Address Flag (R): Not set`, `Valid Lifetime: 86400 (1 day)`, `Preferred Lifetime: 14400 (4 hours)`, `Prefix: fd00:1234:5678::`.

Exemplo de NS/NA capturados (resolução de endereço, c1 procurando o roteador, mesmo par de pacotes visível no Wireshark, frames 3 e 4):

```
fd00:1234:5678:0:88db:a2ff:fe8d:eafd > ff02::1:ff00:1: ICMP6, neighbor solicitation, who has fd00:1234:5678::1, from 8a:db:a2:8d:ea:fd

fd00:1234:5678::1 > fd00:1234:5678:0:88db:a2ff:fe8d:eafd: ICMP6, neighbor advertisement, tgt is fd00:1234:5678::1 (rtr, sol, ovr), is at 9a:75:e2:e2:bd:92
```

Imagem abaixo tela Wireshark com o filtro.



2.5 Conectividade e desempenho (ping, neigh, iperf3):

- Ping c1 → r1 (ULA): 5/5 recebidos, RTT min/avg/max/mdev = 0.057/0.121/0.261/0.074 ms.
- Ping c1 → c2 (ULA): 5/5 recebidos, RTT min/avg/max/mdev = 0.052/0.062/0.087/0.013 ms.

Tabela de vizinhança em c1 após os pings (`ip -6 neigh show dev eth0`):

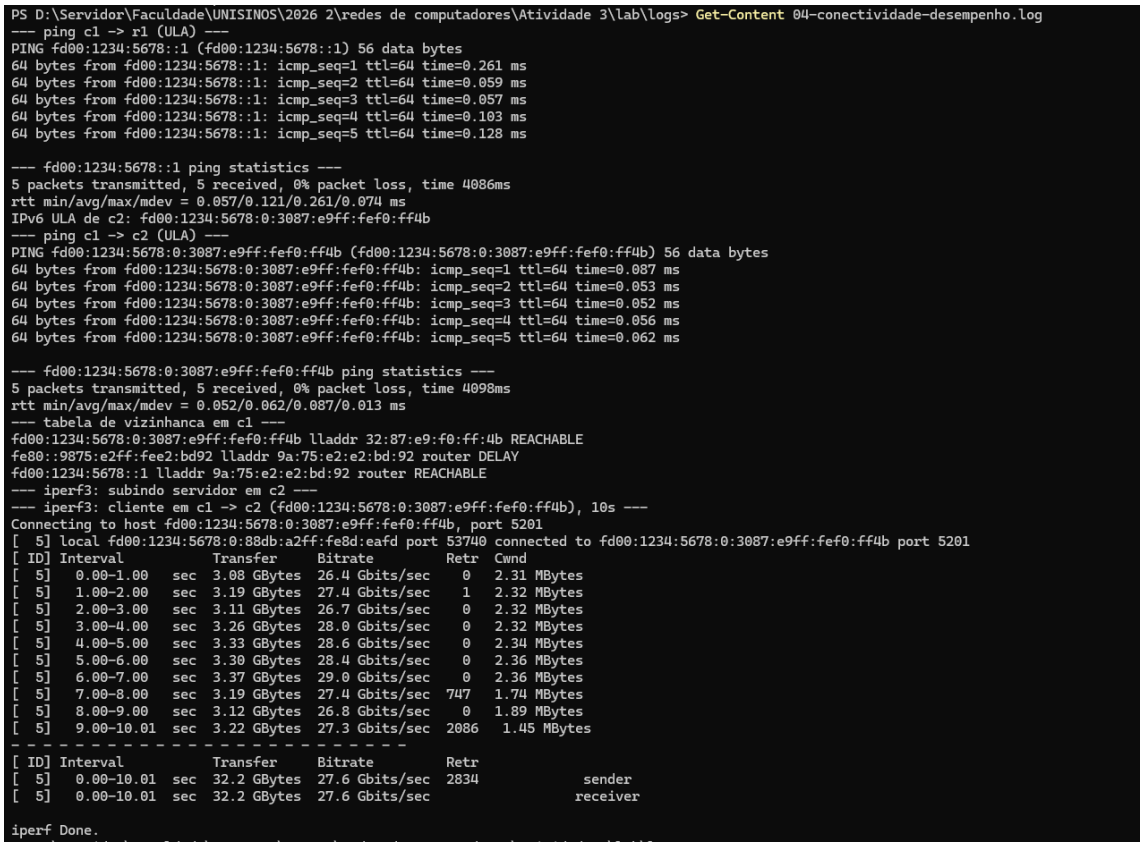
```
fd00:1234:5678:0:3087:e9ff:fef0:ff4b    lladdr 32:87:e9:f0:ff:4b
REACHABLE

fe80::9875:e2ff:fee2:bd92               lladdr 9a:75:e2:e2:bd:92
router DELAY

fd00:1234:5678::1                       lladdr 9a:75:e2:e2:bd:92
router REACHABLE
```

- iperf3 (c1 cliente → c2 servidor, 10s): vazão média 27.6 Gbits/sec, 32.2 GBytes transferidos, 2834 retransmissões no total.

Imagem abaixo, print do terminal com o log dos testes:



```
PS D:\Servidor\Faculdade\UNISINOS\2026 2\redes de computadores\Atividade 3\lab\logs> Get-Content 04-conectividade-desempenho.log
--- ping c1 -> r1 (ULA) ---
PING fd00:1234:5678::1 (fd00:1234:5678::1) 56 data bytes
64 bytes from fd00:1234:5678::1: icmp_seq=1 ttl=64 time=0.261 ms
64 bytes from fd00:1234:5678::1: icmp_seq=2 ttl=64 time=0.059 ms
64 bytes from fd00:1234:5678::1: icmp_seq=3 ttl=64 time=0.057 ms
64 bytes from fd00:1234:5678::1: icmp_seq=4 ttl=64 time=0.103 ms
64 bytes from fd00:1234:5678::1: icmp_seq=5 ttl=64 time=0.128 ms

--- fd00:1234:5678::1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4086ms
rtt min/avg/max/mdev = 0.057/0.121/0.261/0.074 ms
IPv6 ULA de c2: fd00:1234:5678:0:3087:e9ff:fef0:ff4b
--- ping c1 -> c2 (ULA) ---
PING fd00:1234:5678:0:3087:e9ff:fef0:ff4b (fd00:1234:5678:0:3087:e9ff:fef0:ff4b) 56 data bytes
64 bytes from fd00:1234:5678:0:3087:e9ff:fef0:ff4b: icmp_seq=1 ttl=64 time=0.087 ms
64 bytes from fd00:1234:5678:0:3087:e9ff:fef0:ff4b: icmp_seq=2 ttl=64 time=0.053 ms
64 bytes from fd00:1234:5678:0:3087:e9ff:fef0:ff4b: icmp_seq=3 ttl=64 time=0.052 ms
64 bytes from fd00:1234:5678:0:3087:e9ff:fef0:ff4b: icmp_seq=4 ttl=64 time=0.056 ms
64 bytes from fd00:1234:5678:0:3087:e9ff:fef0:ff4b: icmp_seq=5 ttl=64 time=0.062 ms

--- fd00:1234:5678:0:3087:e9ff:fef0:ff4b ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4098ms
rtt min/avg/max/mdev = 0.052/0.062/0.087/0.013 ms
--- tabela de vizinhanca em c1 ---
fd00:1234:5678:0:3087:e9ff:fef0:ff4b lladdr 32:87:e9:f0:ff:4b REACHABLE
fe80::9875:e2ff:fee2:bd92 lladdr 9a:75:e2:e2:bd:92 router DELAY
fd00:1234:5678::1 lladdr 9a:75:e2:e2:bd:92 router REACHABLE
--- iperf3: subindo servidor em c2 ---
--- iperf3: cliente em c1 -> c2 (fd00:1234:5678:0:3087:e9ff:fef0:ff4b), 10s ---
Connecting to host fd00:1234:5678:0:3087:e9ff:fef0:ff4b, port 5201
[  5] local fd00:1234:5678:0:88db:a2ff:fe8d:eafd port 53740 connected to fd00:1234:5678:0:3087:e9ff:fef0:ff4b port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec  3.08 GBytes  26.4 Gbits/sec    0   2.31 MBytes
[  5]   1.00-2.00   sec  3.19 GBytes  27.4 Gbits/sec    1   2.32 MBytes
[  5]   2.00-3.00   sec  3.11 GBytes  26.7 Gbits/sec    0   2.32 MBytes
[  5]   3.00-4.00   sec  3.26 GBytes  28.0 Gbits/sec    0   2.32 MBytes
[  5]   4.00-5.00   sec  3.33 GBytes  28.6 Gbits/sec    0   2.34 MBytes
[  5]   5.00-6.00   sec  3.30 GBytes  28.4 Gbits/sec    0   2.36 MBytes
[  5]   6.00-7.00   sec  3.37 GBytes  29.0 Gbits/sec    0   2.36 MBytes
[  5]   7.00-8.00   sec  3.19 GBytes  27.4 Gbits/sec  747   1.74 MBytes
[  5]   8.00-9.00   sec  3.12 GBytes  26.8 Gbits/sec    0   1.89 MBytes
[  5]   9.00-10.01  sec  3.22 GBytes  27.3 Gbits/sec  2086   1.45 MBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.01  sec  32.2 GBytes  27.6 Gbits/sec  2834             sender
[  5]   0.00-10.01  sec  32.2 GBytes  27.6 Gbits/sec                  receiver

iperf Done.
```

3. Análise do funcionamento do IPv6

1. Endereços link-local de c1 e c2, função do fe80::/10, por que `%eth0` no ping:

c1 = `fe80::88db:a2ff:fe8d:eafd`, c2 = `fe80::3087:e9ff:fef0:ff4b`.

O bloco `fe80::/10` é reservado para comunicação restrita ao link local, não é roteado, e cada interface gera o seu automaticamente ao subir, mesmo sem nenhum roteador ou prefixo global presente. Como esse mesmo prefixo pode se repetir em qualquer link da máquina (cada interface tem seu próprio `fe80::/10`), o endereço sozinho é ambíguo, o SO precisa saber por qual interface (zona/scope) alcançar aquele vizinho. Foi comprovado na prática: pingar `fe80::3087:...` sem `%eth0` falhou com aviso explícito do próprio `ping` ("`may require ifname or scope-id`") e 100% de perda; com `%eth0` funcionou normalmente (RTT min/avg/max = 0.138/0.200/0.263ms).

2. Prefixo anunciado pelo roteador e flags relevantes no RA:

Prefixo `fd00:1234:5678::/64`, capturado no RA com `prefix info option`: `Flags [onlink, auto]`. O flag onlink corresponde a `AdvOnLink` (hosts nesse prefixo são alcançáveis diretamente no link, sem passar por roteador) e auto corresponde a `AdvAutonomous`, que autoriza os hosts a se autoconfigurarem um endereço a partir desse prefixo, é literalmente o flag que liga o SLAAC. O RA também trouxe `router lifetime 1800s` (por quanto tempo r1 deve ser considerado roteador padrão válido).

3. Endereços ULA criados via SLAAC e como confirmamos que não foram manuais:

c1 = `fd00:1234:5678:0:88db:a2ff:fe8d:eafd`, c2 = `fd00:1234:5678:0:3087:e9ff:fef0:ff4b`.

A confirmação não foi "confiar no resultado", o teste foi desenhado para que fosse impossível confundir: o Docker pode gerenciar um subnet IPv6 diferente (`fd00:dead:beef::/64`) do prefixo do laboratório, então qualquer endereço `fd00:1234:5678::/64` só poderia ter vindo do `radvd`. Além disso, o próprio kernel marcou esses endereços com as flags `dynamic mngtmpaddr` no `ip -6 addr show`, a marcação textual padrão do Linux para "obtido por autoconfiguração", ausente no endereço atribuído pelo IPAM do Docker na mesma interface.

4. Função de NS/NA e relação com `ip -6 neigh`:

Neighbor Solicitation é a pergunta "quem tem esse endereço IPv6? me dá seu MAC" (equivalente ao ARP request do IPv4, mas via multicast solicited-node e ICMPv6); Neighbor Advertisement é a resposta com o MAC. Vimos isso literalmente na captura (e conferimos de novo no Wireshark): c1 mandou NS perguntando `who has fd00:1234:5678::1` (de `8a:db:a2:8d:ea:fd`) e r1 respondeu com NA `tgt is fd00:1234:5678::1 (rtr, sol, ovr), is at 9a:75:e2:e2:bd:92`. Cada resolução bem-sucedida é o que povoa uma linha na tabela `ip -6 neigh` — é o cache de "IPv6 → MAC" que evita repetir NS/NA a cada pacote.

5. MACs associados na tabela NDP e estado das entradas:

Tabela em c1 após os testes:

- `fd00:1234:5678::1` (r1, ULA) → MAC `9a:75:e2:e2:bd:92` → REACHABLE
- `fd00:1234:5678:0:3087:...ff4b` (c2, ULA) → MAC `32:87:e9:f0:ff:4b` → REACHABLE
- `fe80::9875:e2ff:fee2:bd92` (r1, link-local) → mesmo MAC do r1 → DELAY

`REACHABLE` = a entrada foi confirmada recentemente (recebemos um NA válido, ou tráfego bidirecional recente). `DELAY` = o kernel tem a entrada mas está aguardando um curto intervalo antes de reconfirmar via NS unicast (transição normal do ciclo de vida NDP, não indica problema).

6. Link-local vs ULA - quando é preciso indicar a interface de saída:

Só é necessário indicar a interface (`%eth0`) quando o destino é link-local (`fe80::/10`), porque esse endereço não é globalmente único, pode existir em qualquer interface de qualquer host na rede, então o SO precisa saber qual zona usar. Com endereço ULA (`fd00:1234:5678::/64`), o endereço já é único dentro daquele prefixo roteável, então o próprio endereço basta para o kernel decidir a rota/interface de saída, não usei `%eth0` nos pings por ULA e funcionaram normalmente.

7. Latência do ping e coerência com rede virtual no mesmo host:

c1→r1: média 0.121ms; c1→c2: média 0.062ms, na faixa de dezenas/centenas de microssegundos. É totalmente coerente com uma rede 100% virtual rodando no mesmo host: não existe NIC física, cabo ou switch, os pacotes trafegam por uma bridge de software dentro do próprio kernel Linux (no caso via WSL2), então o "meio físico" é essencialmente cópia de memória.

8. Vazão do iperf3 e fatores que podem ter influenciado:

27.6 Gbits/sec médios (32.2 GBytes em mais ou menos 10s), com 2834 retransmissões no total (uma fração pequena frente ao volume total transferido, sem impacto perceptível na vazão média). Esse número é muito mais alto do que qualquer enlace físico real (mesmo 10G) permitiria, o que confirma que a "rede" aqui é limitada por CPU/memória do host (Windows rodando Docker Desktop sobre WSL2), não por um meio de transmissão de verdade. Fatores que influenciam: poder de processamento do host, overhead da própria virtualização (bridge de software, tradução de rede do WSL2), e o MTU (1500, padrão, um MTU maior, como jumbo frames, tenderia a reduzir overhead de cabeçalho por byte transferido e aumentar ainda mais a vazão medida).

4. Análise crítica do ambiente

O comportamento observado era o esperado, com uma ressalva importante. A mecânica central do IPv6 (link-local automático, RA anunciando prefixo, SLAAC compondo o endereço, NDP resolvendo MACs) funcionou exatamente como a teoria descreve. A ressalva é que "funcionar" não foi automático: exigiu decisões explícitas de configuração para garantir que o que estávamos observando era de fato SLAAC, e não um mecanismo diferente disfarçado de SLAAC pela camada de virtualização. Isso por si só já é uma lição: na teoria, SLAAC "simplesmente acontece"; na prática, dentro de um ambiente com múltiplas camadas de rede (host Windows, WSL2, Docker, container), é preciso entender cada camada para saber se o que se está vendo é o protocolo real ou um efeito colateral da ferramenta.

Influência da virtualização, bridge, containers e MTU. Rodamos tudo sobre Docker Desktop com backend WSL2 em um host Windows, ou seja, entre o "roteador" r1 e os "clientes" c1/c2 não existe nenhum meio físico: os pacotes trafegam por uma bridge de software dentro do mesmo kernel Linux (o do WSL2). Isso explica diretamente dois resultados que só fazem sentido à luz dessa arquitetura: a latência do ping ficou na casa de 0.05 a 0.25ms (muito abaixo do que se vê em uma LAN física real, porque não há cabo, switch ou NIC física envolvidos, é essencialmente cópia de memória) e a vazão do iperf3

(27.6 Gbits/sec) é claramente limitada por CPU/memória do host, não por um link de rede real, já que nenhuma tecnologia de rede física doméstica chega perto disso. O MTU (1500, padrão do driver bridge do Docker) também influencia: é o mesmo valor usado em Ethernet física, então não introduziu fragmentação artificial nos testes, mas um MTU maior tenderia a reduzir overhead de cabeçalho por byte e aumentar ainda mais a vazão medida, um lembrete de que "desempenho de rede" em ambiente virtualizado mede mais a eficiência da pilha de software do que qualquer coisa parecida com uma rede real.

SLAAC vs. atribuição de endereço por ferramenta de virtualização (IPAM do Docker). Essa foi a descoberta central do experimento. Por padrão, se a rede Docker for criada com IPv6 habilitado usando o mesmo prefixo do laboratório, o Docker atribui um endereço a cada container via seu próprio IPAM, um mecanismo stateful, decidido centralmente pelo Docker, sem nenhuma mensagem RA ou NS envolvida. Isso pareceria "SLAAC" superficialmente (é um endereço IPv6 global que "simplesmente aparece"), mas não é: não existe autoconfiguração nenhuma ali, é atribuição direta, análoga a um DHCP muito simplificado. Para provar SLAAC de verdade, foi usado deliberadamente um prefixo diferente para o IPAM do Docker (fd00:dead:beef::/64) do prefixo do laboratório (fd00:1234:5678::/64, anunciado pelo radvd). O resultado foi cada container ficar com dois endereços globais simultâneos e visivelmente diferentes: um sem marcação especial (IPAM) e outro com as flags dynamic mngtmpaddr (SLAAC), a prova textual do próprio kernel de que aquele endereço específico veio de autoconfiguração via RA.

O papel do ICMPv6 para RA, NDP e o funcionamento correto do IPv6. No IPv4, o ICMP é essencialmente auxiliar, usado para diagnóstico (ping) e mensagens de erro, e o endereçamento depende de outro protocolo inteiro (o DHCP) ou de configuração manual. No IPv6, o ICMPv6 deixa de ser auxiliar e passa a ser infraestrutura, é ele quem carrega o Router Advertisement (que viabiliza o SLAAC) e o Neighbor Solicitation/Advertisement, que substitui o ARP do IPv4 por completo. Sem ICMPv6 funcionando corretamente, não existe descoberta de vizinhos, não existe autoconfiguração e, na prática, a rede IPv6 simplesmente não opera, algo que fica muito claro ao ver, na captura, que a

primeira coisa que acontece na rede (antes de qualquer ping do usuário) são pacotes ICMPv6 de RA e, no primeiro ping, um par de NS/NA resolvendo o MAC do destino antes do echo request sequer ser enviado.

O que a captura de pacotes revelou que a teoria sozinha não mostra. Ler que "o SLAAC usa RA para autoconfiguração" é uma frase, ver na captura, o RA chegando com as flags onlink e auto e o endereço em c1 aparecendo na sequência imediata com exatamente esse prefixo é outra coisa, a causalidade fica visível, não é mais uma afirmação abstrata. O mesmo vale para a tabela de vizinhança, a teoria descreve os estados NDP (REACHABLE, STALE, DELAY, etc.) como uma lista, a captura mostrou o estado DELAY aparecendo naturalmente durante a transição normal do ciclo de vida de uma entrada, sem que precisássemos forçar nenhum cenário de erro, algo que só se observa rodando o experimento de verdade. Por fim, a existência simultânea de dois endereços globais por interface (IPAM do Docker mais SLAAC) só ficou evidente com a captura e com o comando ip -6 addr show lado a lado.

Reflexão final. Um ponto que vale registrar, a complexidade do IPv6 frente ao IPv4. O espaço de endereçamento de 2^128 (contra 2^32 do IPv4) e mecanismos como SLAAC/NDP foram desenhados para resolver de forma estrutural problemas que o IPv4 remenda com NAT. Mas essa mesma robustez tem um custo de complexidade operacional visível neste próprio laboratório, precisamos entender forwarding, accept_ra, flags de RA, sysctls por interface, nada disso tem equivalente simples em uma rede IPv4 doméstica típica, onde o DHCP resolve tudo de forma opaca. Isso ajuda a explicar por que a adoção do IPv6 foi tão mais lenta do que se esperava. O NAT deu sobrevida ao IPv4 ao mascarar a escassez de endereços, removendo a urgência, enquanto a migração para IPv6 exige lidar com essa complexidade adicional em toda a pilha de rede, do roteador ao host final.

Sobre a arquitetura técnica escolhida (Docker e sysctls): SLAAC dentro do Docker exige cuidado. Por padrão, se você deixa o Docker gerenciar o IPv6 da rede (IPAM) usando o mesmo prefixo do laboratório, o endereço que aparece no container não é produto de SLAAC, é preciso decisão explícita (rede com prefixo separado do IPAM mais radvd real) para reproduzir o mecanismo de verdade. Isso é uma diferença prática relevante frente ao

ambiente de referência do roteiro (namespaces puros), onde não existe essa camada extra de gerência de endereços. Sysctls em containers também são restritos por segurança. O Docker só permite alterar em runtime (sysctl -w) os parâmetros que já foram declarados no docker-compose.yml na subida do container; os demais retornam erro de sistema de arquivos somente leitura. Isso molda como a configuração precisa ser feita, de forma declarativa, na definição do serviço, não improvisada depois. Por fim, forwarding fica habilitado apenas no roteador. Nos clientes, forwarding com valor 1 faz o kernel ignorar RA por padrão, a menos que accept_ra igual a 2 force o aceite.

5. Diário de bordo / uso de Inteligência Artificial

Ferramenta utilizada: Claude (Anthropic), via Claude Code.

18-08-2026, Planejamento inicial. Descrevi ao Claude a ideia de usar Docker Compose com 3 containers simulando r1/c1/c2, scripts de configuração e um sistema de logs. Como subir 3 VMs exige recurso computacional que não tenho, pensei em usar Docker mesmo. Claude analisou e levantou o mesmo problema que o professor sobre usar Docker nesse experimento por conta do Docker atribuir os IPs (como já descrito anteriormente). Ele me propôs alguns testes antes de iniciarmos.

Teste mínimo (r1 + c1), erros reais encontrados. Ao rodar radvd -C /etc/radvd.conf via docker compose exec no terminal (Git Bash, Windows), o radvd falhou com "can't open C:/Program Files/Git/etc/radvd.conf: No such file or directory". O Git Bash estava reescrevendo o path Unix /etc/radvd.conf como se fosse um path do Windows. Precisei confirmar a causa e corrigir com a variável de ambiente MSYS_NO_PATHCONV=1 antes do comando, um problema de ferramenta/ambiente Windows, que teria travado toda a atividade se não fosse identificado. Também, ao tentar sysctl -w net.ipv6.conf.all.forwarding=1 manualmente dentro do container já em execução, recebi "Read-only file system". Investigamos e o valor já estava correto (1), porque tinha sido setado na definição do docker-compose.yml (sysctls) desde a subida do container, o Docker bloqueia alteração posterior de

sysctls não declarados antecipadamente. Foi validado lendo diretamente /proc/sys/net/ipv6/conf/all/forwarding.

Validação do resultado. Conferimos que o endereço IPv6 global criado em c1 (fd00:1234:5678:...) carregava as flags dynamic mngtmpaddr no ip -6 addr show, essa é a evidência textual do kernel de que o endereço veio de autoconfiguração via RA, e não do IPAM do Docker (que atribuiu, na mesma interface, um endereço fd00:dead:beef::2 sem essas flags). Sem essa checagem específica, teríamos corrido o risco de reportar um "SLAAC" que na verdade era o Docker atribuindo endereço por conta própria, exatamente o erro que o professor avisa no roteiro.

Checkpoint pedagógico. Antes de sair replicando esse accept_ra: 2 para todos os serviços "por garantia", como eu tinha dito para irmos construindo juntos e que ele não era para fazer tudo sozinho, o Claude me perguntou por que eu tinha colocado esse sysctl no c1, já que o c1 nunca teve forwarding ligado. Minha resposta foi " só por precaução, mas não tenho certeza se realmente precisava". Estava certo: o valor padrão do Linux (accept_ra=1) já aceita RA normalmente desde que forwarding esteja desligado, o valor 2 só muda alguma coisa quando forwarding=1 no mesmo host, forçando aceitar RA mesmo assim (por padrão, um host com forwarding ligado passa a ignorar RA, sem nenhum erro visível, um bug bem difícil de debugar se você não souber dessa regra). Ou seja, não estava errado tecnicamente, mas também não sabia exatamente por quê funcionava, bom exemplo de "acertei no chute, entendi depois".

18-08-2026, Evolução para a topologia completa (r1 + c1 + c2). Depois do teste mínimo validado, montamos a estrutura final em lab/ (Dockerfile, docker-compose.yml com os três serviços, e o script run-lab.sh que executa e loga cada etapa do roteiro em ordem).

Erro novo, ambiente Windows. Ao rodar run-lab.sh, exportei MSYS_NO_PATHCONV=1 para evitar que o Git Bash reescrevesse paths internos dos containers (como /scripts/setup-router.sh) como se fossem paths do Windows. Só que isso quebrou o argumento -f com o path do docker-compose.yml, que é um path do host e precisa ser convertido, o erro foi o docker compose tentando abrir "D:\d\Servidor..." (o Git Bash, com a conversão desligada, passou o path Unix literal e o Docker interpretou o "/d/" inicial como

uma pasta chamada "d", não como a letra da unidade). Corrigido trocando o -f com path absoluto por um simples cd na pasta antes de chamar docker compose (que descobre o docker-compose.yml sozinho no diretório atual), assim só os argumentos que realmente precisam ficar crus (paths dentro do container) ficam sob o MSYS_NO_PATHCONV, e o path do host nunca passa por linha de comando.

Resultado. Rodada completa executada com sucesso: SLAAC confirmado em c1 e c2 simultaneamente, ping por ULA e link-local (%eth0) testados, tabela de vizinhança com MACs reais capturada, arquivo pcap com RA/NS/NA salvo, iperf3 rodou (28.1 Gbits/sec). Os dados reais já estão preenchidos nas seções 1 a 3 deste relatório.

Nota de rodapé (não tão séria, mas real). O subnet escolhido para o IPAM do Docker, fd00:dead:beef::/64: 0xDEADBEEF é um valor hexadecimal clássico usado desde os anos 70/80 como placeholder de debug (fácil de reconhecer visualmente em um dump de memória). Usado aqui de propósito, tanto pela referência quanto porque facilita diferenciar visualmente do prefixo real do laboratório (fd00:1234:5678) nos comandos e capturas, sugestão do Claude e eu adorei a piada.

22-08-2026, Prints, Wireshark e análise crítica. Depois de alguns dias, retomei o trabalho pra fazer os prints de tela. Os containers Docker tinham reiniciado nesse meio tempo (perdendo o endereço ULA manual do r1 e o processo do radvd, que não são persistidos entre reinícios), reexecutei run-lab.sh do zero pra gerar uma captura e logs frescos e consistentes entre si antes de tirar qualquer print, evitando misturar dados de rodadas diferentes no mesmo relatório.

Usei o Wireshark pela primeira vez nessa atividade. O filtro de exibição usado (icmpv6.type == 134 || 135 || 136) é o mesmo já sugerido no roteiro do professor. A partir daí, o Claude sugeriu que eu explorasse sozinho o pacote de Router Advertisement para conferir se as flags on-link/autonomous batiam com o que já tínhamos visto via linha de comando (tcpdump -vv), bati, e de quebra descobri (e o Claude corrigiu um ponto que eu tinha entendido errado) que o hop limit do IPv6 é o equivalente ao TTL do IPv4, mas os campos Valid/Preferred Lifetime da opção de prefixo são uma coisa totalmente diferente

(validade do endereço autoconfigurado, não contagem de saltos), eu tinha misturado os dois conceitos por soarem parecido ("tempo de vida").

Na escrita da análise crítica (seção 4), errei uma unidade que o Claude corrigiu: eu tinha lido o resultado do iperf3 como "27.6 GB/s" quando na verdade é 27.6 Gbits/s (gigabits, não gigabytes), uma diferença de 8 vezes que teria ficado errada no relatório final se não fosse verificada contra o dado bruto do log.

Qual ferramenta de IA foi utilizada: Claude (Anthropic), via Claude Code, usado durante todas as etapas.

Em quais etapas ela ajudou: análise crítica do plano inicial, identificando o risco de a abordagem em Docker cair na armadilha do IPAM mascarando SLAAC (avisada pelo próprio roteiro do professor); validação no desenho e implementação da infraestrutura (Dockerfile, docker-compose.yml, radvd.conf, scripts de orquestração e logging); acompanhamento e depuração dos testes (incluindo os dois erros de ambiente Windows registrados acima); explicações conceituais sob demanda (o que é SLAAC que eu não me lembrava, diferença entre hop limit e lifetime, etc.), sempre amarradas ao que estava sendo observado no experimento; e redação assistida das respostas da seção 3 e da análise crítica (seção 4), a partir dos dados reais coletados e da minha própria reflexão sobre o experimento.

Como validei as sugestões da IA com dados reais do experimento: nenhuma afirmação técnica foi aceita porque a IA disse, cada uma foi checada contra o comportamento real do sistema: os endereços SLAAC foram confirmados pelas flags dynamic mngtmpaddr do próprio kernel, o comportamento do %eth0 foi testado nos dois sentidos (com e sem) pra ver o erro real acontecer e os flags do RA foram conferidos tanto via tcpdump -vv quanto depois, de forma independente, abrindo o pcap no Wireshark manualmente.

Exemplo de sugestão que precisei confirmar ou corrigir: o sysctl accept_ra: 2 que eu tinha colocado no c1 "por garantia", quando questionado sobre o motivo, percebi que não sabia explicar com precisão. A explicação correta (RA é ignorado por padrão quando forwarding=1, e o valor 2 força o

aceite) só ficou clara depois de eu tentar responder sozinho e ser corrigido, registrado em detalhe no "Checkpoint pedagógico" acima. Outro exemplo: a unidade errada que usei na análise crítica (GB/s em vez de Gbit/s), corrigida antes de entrar na versão final do relatório.